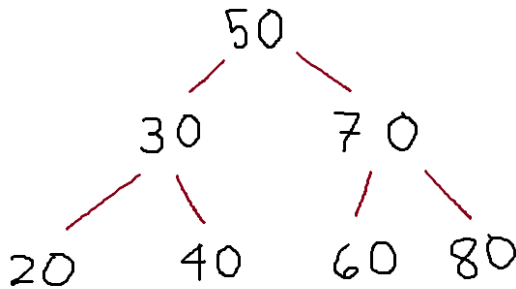


1. Dibujar el método e identifica



Raíz: 50

Hijos de 50: 30 (izquierdo) y 70 (derecho)

Hijos de 30: 20 (izquierdo) y 40 (derecho)

Hijos de 70: 60 (izquierdo) y 80 (derecho)

Nodos hoja: 20, 40, 60 y 80

2. ¿Qué propiedad debe cumplir todo Árbol Binario de Búsqueda?
Todo árbol binario debe cumplir que por cada nodo los valores menores están en su subárbol izquierdo y los mayores a la derecha
3. ¿Cuál es la raíz del árbol construido?
50
4. ¿Qué nodos son hojas?
20,40,60,80
5. ¿Qué valores pertenecen al subárbol izquierdo de 50 y cuáles al derecho?
Subarbol izquierdo:
30,20,40
Subarbol derecho:
70,60,80
6. ¿Qué secuencia esperas obtener con el recorrido inorden?
20,30,40,50,60,70,80

7. Metodo de búsqueda

- a) ¿Por qué no es necesario recorrer todos los nodos del árbol para buscar una clave?
Porque el árbol nos dice hacia donde tenemos que ir ya que si buscamos un numero menor que el nodo en el que estamos, sabemos que hay que ir a la izquierda y si es mayor nos vamos a la derecha
- b) Si se busca 40, ¿qué nodos se visitan y en qué orden?
50→30→40
- c) Si se busca 90, ¿qué condición permitirá concluir que no existe?
Cuando recorremos el árbol llega un punto en el que acabamos en null, esto significa que ya no hay ningún nodo a donde seguir buscando
- d) ¿Qué valor booleano debe regresar el caso base cuando el nodo actual es null?
Debe regresar false
- e) ¿Qué ocurriría si el árbol no respetara la regla menor-izquierda y mayor-derecha?
La búsqueda ya no funcionaria correctamente porque ya no sabríamos si hay que ir a la izquierda o a la derecha

8. Metodo de eliminación

- a) ¿Por qué la eliminación requiere más casos que la búsqueda?
Porque buscar solo necesita ver si existe, en cambio, en eliminar se necesita mantener la estructura del árbol
- b) ¿Qué debe ocurrir si la clave que se desea eliminar no existe?
Devuelve null y no modifica el árbol
- c) ¿Por qué eliminar un nodo hoja es el caso más sencillo?
Porque no tiene hijos, solo se elimina y ya
- d) Si un nodo tiene solamente un hijo, ¿por qué puede devolverse directamente la referencia a ese hijo?
Porque ese hijo puede ocupar el lugar del nodo eliminado sin romper la estructura del árbol

- e) ¿Por qué el menor valor del subárbol derecho es un candidato adecuado para sustituir a un nodo con dos hijos?
Porque es el siguiente valor más grande después del nodo eliminado y mantiene el orden del árbol
- f) Después de copiar el valor sustituto, ¿por qué todavía es necesario eliminar ese valor de su ubicación original?
Porque si solo se copiara el valor, se tendría el mismo valor dos veces en el árbol
- g) ¿Qué riesgo existiría si se eliminara un nodo con dos hijos sin reconectar correctamente sus subárboles?
Se podría perder una rama completa del árbol
- h) ¿Por qué eliminar la raíz puede modificar la variable raíz del árbol?
Porque si lo eliminamos, otro nodo tiene que agarrar su lugar
- i) ¿Qué propiedad debe seguir cumpliendo el árbol después de cualquier eliminación?
Debe cumplir que los menores a la izquierda y los mayores a la derecha

9. Metodo auxiliar

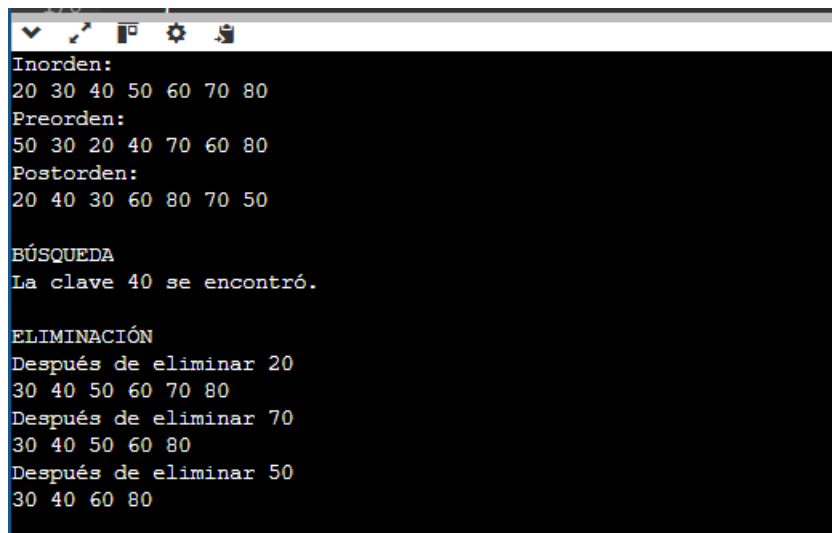
- a) ¿Hacia qué dirección debes desplazarte para encontrar el mínimo?
Izquierda
- b) ¿Qué condición indica que ya encontraste el nodo mínimo?
Raíz.izquierdo==null, esto se da cuando el nodo ya no tiene hijos
- c) ¿Cuál es el mínimo del subárbol cuya raíz es 70 en el árbol inicial?
El mínimos es 60

10. Reflexiones finales

- a) ¿Cómo ayuda el recorrido inorden a comprobar que el ABB conserva su estructura?
El recorrido inorden deja comprobar que los valores siguen apareciendo de menor a mayor. Si después de insertar o eliminar elementos el resultado continúa ordenado, significa que se está conservando la propiedad del arbol

- b) ¿Cuál fue el caso de eliminación más difícil?
El caso más difícil fue eliminar un nodo con dos hijos, porque no se puede simplemente borrar el nodo. Primero se debe buscar un valor que lo sustituya y después eliminar ese valor de su posición original.
- c) ¿Qué papel cumple la recursividad en los métodos de búsqueda y eliminación?
La recursividad permite avanzar por el árbol sin tener que recorrerlo completo.
- d) ¿Qué aprendiste sobre el cambio de referencias entre nodos al eliminar elementos?
Aprendí que al eliminar un nodo no siempre desaparece simplemente, sino que sus hijos pueden tener que ocupar su lugar.
- e) ¿Cuál es la diferencia entre buscar y eliminar en un ABB?
Buscar consiste en recorrer el árbol hasta encontrar una clave o llegar a null. Eliminar es más complejo porque, además de encontrar la clave, hay que modificar las referencias para mantener la estructura y las propiedades del árbol.

11. Capturas:



```
Inorden:
20 30 40 50 60 70 80
Preorden:
50 30 20 40 70 60 80
Postorden:
20 40 30 60 80 70 50

BÚSQUEDA
La clave 40 se encontró.

ELIMINACIÓN
Después de eliminar 20
30 40 50 60 70 80
Después de eliminar 70
30 40 50 60 80
Después de eliminar 50
30 40 60 80
```